

Taller de demostración guiada de sesgos en python

Juan Pablo Florez Soto

Juan Manuel Franco

Milton Vanegas Delgado

Unidad de sistemas y electricidad, UNINTEP

8vo Semestre, Ingeniería Informática

Inteligencia Artificial

Oscar Fernando Pulgarín

18 de septiembre de 2026

Taller de demostración guiada de sesgos en python

Reflexión ética y técnica: sesgo en el dataset bank marketing

Cargar la fuente

```
SCRIPT_DIR = Path(__file__).resolve().parent
BASE_DIR = SCRIPT_DIR.parent.parent

RAW_DIR = BASE_DIR / "datos"

FILE_ADULT = RAW_DIR / "bank-full.csv" #Directorio local donde está
alojado el archivo en texto plano.
```

Salida de consola

```
  age      job marital education default balance housing loan \
0  58  management married tertiary    no   2143    yes  no
1  44  technician single secondary    no    29    yes  no
2  33  entrepreneur married secondary    no     2    yes yes
3  47  blue-collar married   unknown    no  1506    yes  no
4  33    unknown single    unknown    no     1    no  no

  contact day month duration campaign pdays previous poutcome y
0 unknown   5 may     261         1   -1     0 unknown no
1 unknown   5 may     151         1   -1     0 unknown no
2 unknown   5 may     76         1   -1     0 unknown no
3 unknown   5 may     92         1   -1     0 unknown no
4 unknown   5 may    198         1   -1     0 unknown no
```

Paso 1 — Exploración inicial (EDA)

```
df = pd.read_csv(FILE_ADULT, sep=";")
print(df.head())
print("\nEstructura:", df.shape)
print("\n#####Información
General#####")
df.info()

# Inspección de unknown por columna contact
print("\nValores unknown detectado por columna:")
columnas_categoricas = df.select_dtypes(include="object").columns
for col in columnas_categoricas:
    n_unknown = (df[col] == "unknown").sum()
```

```

    if n_unknown > 0:
        pct = n_unknown / len(df) * 100
        print(f"{col}: {n_unknown} valores 'unknown' ({pct:.2f}%)")

df["age"].describe()

print("\nDistribución de la variable sensible: Estado Civil\n",
df["marital"].value_counts(normalize=True) * 100)
print("\nDistribución de la variable objetivo: Suscripción\n",
df["y"].value_counts(normalize=True) * 100)
print("\nEstadísticas de edad\n", df["age"].describe())

```

Salida de consola

Estructura: (45211, 17)

#####Información General#####

<class 'pandas.DataFrame'>

RangeIndex: 45211 entries, 0 to 45210

Data columns (total 17 columns):

#	Column	Non-Null Count	Dtype
0	age	45211 non-null	int64
1	job	45211 non-null	str
2	marital	45211 non-null	str
3	education	45211 non-null	str
4	default	45211 non-null	str
5	balance	45211 non-null	int64
6	housing	45211 non-null	str
7	loan	45211 non-null	str
8	contact	45211 non-null	str
9	day	45211 non-null	int64
10	month	45211 non-null	str
11	duration	45211 non-null	int64
12	campaign	45211 non-null	int64
13	pdays	45211 non-null	int64
14	previous	45211 non-null	int64
15	poutcome	45211 non-null	str
16	y	45211 non-null	str

dtypes: int64(7), str(10)

memory usage: 5.9 MB

Valores unknown detectado por columna:

D:\Usuarios-PC\WebMaster\Documents\InteligenciaArtificial\src\Analisis\TallerSesgos.py:40: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this

```

warning.
See
https://pandas.pydata.org/docs/user\_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
columnas_categoricas = df.select_dtypes(include="object").columns
job: 288 valores 'unknown' (0.64%)
education: 1857 valores 'unknown' (4.11%)
contact: 13020 valores 'unknown' (28.80%)
poutcome: 36959 valores 'unknown' (81.75%)

Distribución de la variable sensible: Estado Civil
marital
married    60.193316
single    28.289576
divorced   11.517109
Name: proportion, dtype: float64

Distribución de la variable objetivo: Suscripción
y
no    88.30152
yes   11.69848
Name: proportion, dtype: float64

Estadísticas de edad
count    45211.000000
mean     40.936210
std      10.618762
min      18.000000
25%      33.000000
50%      39.000000
75%      48.000000
max      95.000000
Name: age, dtype: float64

```

1. ¿Algún grupo demográfico está claramente sub-representado en el número de registros?

Sí, en las dos variables sensibles del taller. En estado civil, divorced representa solo 11.52% (5.207 filas) frente a 60.19% de married y 28.29% de single es el grupo claramente minoritario. En edad, al agrupar en bins, los extremos quedan muy subrepresentados: 18 - 25 años es apenas 2.85% (1.231 filas) y 66+ es solo 1.55% (670 filas), mientras que 26 - 35 y 36-45 concentran juntos el 65.9% del dataset. Estos dos subgrupos pequeños (18 - 25 y 66+) son precisamente los que, como se ve en el Paso 3, muestran las tasas de suscripción más extremas, lo cual exige cautela: con muestras de 670 - 1231 registros, esas tasas tienen más varianza y menos estabilidad estadística que las de los grupos centrales.

Paso 2 — Limpieza consciente del sesgo

```
cols_faltantes_genuinos = ["job", "education"]
# Ver el impacto de eliminar filas con nulos, ANTES de hacerlo
antes = df["marital"].value_counts(normalize=True) * 100
df_na = df.copy()
df_na[cols_faltantes_genuinos] =
df_na[cols_faltantes_genuinos].replace("unknown", np.nan)
df_limpio = df_na.dropna(subset=cols_faltantes_genuinos).copy()

despues = df_limpio["marital"].value_counts(normalize=True) * 100

comparacion = pd.DataFrame({"Antes de limpiar (%)": antes, "Después
de limpiar (%)": despues, "Diferencia": despues - antes})
print("\nImpacto de de la limpieza en la variable marital (Estado
Civil):\n",comparacion.round(2))

filas_antes, filas_despues = len(df), len(df_limpio)
pct_perdido = (1 - filas_despues / filas_antes) * 100
print(f"\nTotal de filas ANTES: {filas_antes}")
print(f"Total de filas DESPUÉS: {filas_despues}")
print(f"Porcentaje de filas descartadas: {pct_perdido:.2f}%")

bins_edad = [0, 25, 35, 45, 55, 65, 100]
etiquetas_edad = ["18-25", "26-35", "36-45", "46-55", "56-65", "66+"]
df_limpio["grupo_edad"] = pd.cut(df_limpio["age"], bins=bins_edad,
labels=etiquetas_edad)

print("\n--- Distribución por grupo de edad (después de limpiar)
---")
print(df_limpio["grupo_edad"].value_counts(normalize=True).sort_index
() * 100)

print("\n--- Tamaño real de cada subgrupo de edad (n) ---")
print(df_limpio["grupo_edad"].value_counts().sort_index())
```

Salida de consola

Impacto de de la limpieza en la variable marital (Estado Civil):			
	Antes de limpiar (%)	Después de limpiar (%)	Diferencia
marital			
married	60.19	60.07	-0.12
single	28.29	28.29	-0.00

```

divorced          11.52          11.64          0.12

Total de filas ANTES: 45211
Total de filas DESPUÉS: 43193
Porcentaje de filas descartadas: 4.46%

--- Distribución por grupo de edad (después de limpiar) ---
grupo_edad
18-25    2.849999
26-35   35.153844
36-45   30.743407
46-55   20.774200
56-65    8.927372
66+     1.551177
Name: proportion, dtype: float64

--- Tamaño real de cada subgrupo de edad (n) ---
grupo_edad
18-25    1231
26-35   15184
36-45   13279
46-55    8973
56-65    3856
66+      670
Name: count, dtype: int64

```

2. Si dropna() cambia notablemente la proporción de un grupo, la limpieza introdujo sesgo adicional. ¿Conviene imputar en vez de eliminar, y por qué?

En este caso el corrimiento es mínimo: married baja 0.12 puntos y divorced sube 0.12 puntos, single no se mueve. Esto sugiere que los 'unknown' en job y education no están fuertemente correlacionados con marital, por lo que eliminar esas 2.018 filas (4.46% del total) es una decisión razonable y de bajo riesgo para la variable estado civil.

Aun así, el criterio general del taller sigue aplicando para justificar por qué no se generalizó dropna() a todo el dataset: contact (28.80% unknown) y poutcome (81.75% unknown) se dejaron intactos a propósito, porque eliminarlas habría destruido la mayoría del dataset y, como se explica en la revisión de código, poutcome="unknown" no es aleatorio sino estructural (clientes nunca contactados antes). Conviene imputar en vez de eliminar cuando:

- El porcentaje de faltantes es alto (como en contact y poutcome).
- El faltante tiene un significado propio que no es realmente 'ausencia de dato'
- Eliminar reduciría demasiado un subgrupo ya pequeño (como 66+, con solo 670 registros: perder unos pocos ahí pesa mucho más que perder los mismos registros en 26-35).

Paso 3 — Tasa de resultado positivo por grupo (paridad estadística)

```

df["suscrito"] = (df["y"] == "yes").astype(int)
df_limpio["suscrito"] = (df_limpio["y"] == "yes").astype(int)

#Función tasa positiva
def tasa_positiva(data, columna_grupo):
    resumen = data.groupby(columna_grupo,
observed=True)["suscrito"].agg(["mean", "count"])
    resumen["mean"] = resumen["mean"] * 100
    resumen.columns = ["tasa_%", "n"]
    return resumen

tasa_estado_civil = tasa_positiva(df_limpio, "marital")
tasa_grupo_edad = tasa_positiva(df_limpio, "grupo_edad")

print("\nTasa de suscripción por estado civil:", tasa_estado_civil)
print("\nTasa de suscripción por grupo de edad:", tasa_grupo_edad)

disparidad_estado_civil = tasa_estado_civil["tasa_%"].max() -
tasa_estado_civil["tasa_%"].min()
disparidad_edad = tasa_grupo_edad["tasa_%"].max() -
tasa_grupo_edad["tasa_%"].min()

print(f"\nDisparidad por ecstado civil: {disparidad_estado_civil:.2f}
puntos porcentuales")
print(f"Disparidad por grupo de edad: {disparidad_edad:.2f} puntos
porcentuales")

tasa_global_completo = df["suscrito"].mean() * 100
tasa_global_limpio = df_limpio["suscrito"].mean() * 100
print(f"\nTasa de suscripción global – dataset completo
(n={len(df)}): {tasa_global_completo:.2f}%")
print(f"Tasa de suscripción global – dataset limpio
(n={len(df_limpio)}): {tasa_global_limpio:.2f}%")

```

Salida de consola

```

Tasa de suscripción por estado civil:      tasa_%    n
marital
divorced  11.893397  5028
married  10.032375  25946
single   14.894836  12219

```

```

Tasa de suscripción por grupo de edad:      tasa_%    n

```

grupo_edad		
18-25	23.314379	1231
26-35	11.992887	15184
36-45	9.300399	13279
46-55	9.316839	8973
56-65	14.341286	3856
66+	43.134328	670

Disparidad por estado civil: 4.86 puntos porcentuales
 Disparidad por grupo de edad: 33.83 puntos porcentuales

Tasa de suscripción global — dataset completo (n=45211): 11.70%
 Tasa de suscripción global — dataset limpio (n=43193): 11.62%

3. Valores por encima de ~0.1–0.2 suelen considerarse una señal de alerta en la literatura de fairness. ¿Qué significa esto?

Aplicado a estos datos, el umbral separa dos historias muy distintas dentro del mismo dataset. La disparidad por estado civil (0.0486) queda por **debajo** del umbral de alerta married, single y divorced se suscriben a tasas relativamente parecidas (10 - 15%), y esa diferencia probablemente no amerita una intervención de fairness prioritaria. La disparidad por grupo de edad (0.3383), en cambio, supera el umbral con amplio margen: 66+ se suscribe 43.13% de las veces frente a 9.30% en 36 - 45, una brecha de más de 4 a 1.

El umbral no dice automáticamente que hay discriminación, en este dataset la explicación más plausible es que jubilados y estudiantes tienen perfiles financieros y de tiempo disponible distintos que los hacen más receptivos a depósitos a plazo pero sí exige tratar age como una variable que un modelo predictivo puede aprender a explotar de forma desigual, y que merece auditoría si se usa para decisiones (por ejemplo, a quién priorizar en campañas de marketing).

Paso 4 — Visualización comparativa

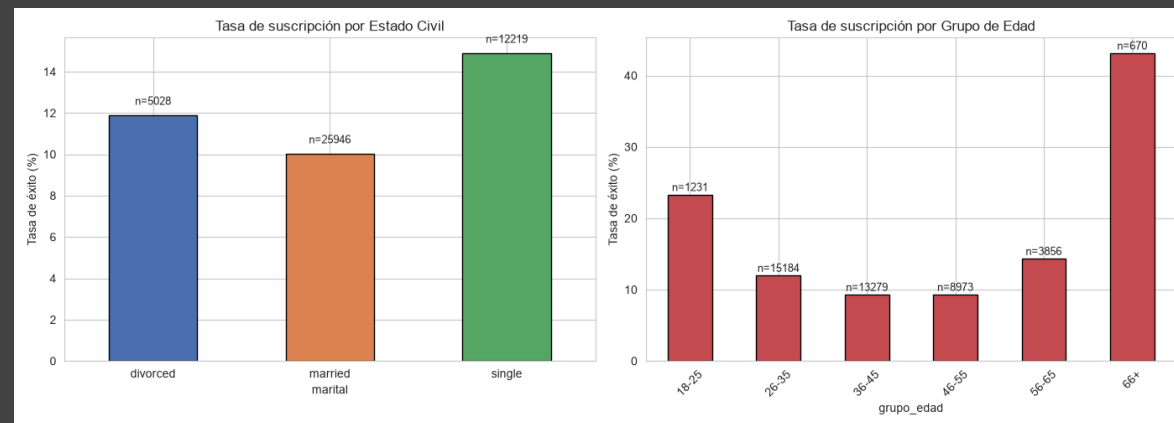
```
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
#Tasa por estado civil (Panel izquierdo)
tasa_estado_civil["tasa_%"].plot(kind="bar", ax=ax[0],
color=["#4C72B0", "#DD8452", "#55A868"], edgecolor="black")
ax[0].set_title("Tasa de suscripción por Estado Civil", fontsize=12)
ax[0].set_ylabel("Tasa de éxito (%)")
ax[0].tick_params(axis="x", rotation=0)
for i, (tasa, n) in enumerate(zip(tasa_estado_civil["tasa_%"],
tasa_estado_civil["n"])):
    ax[0].text(i, tasa + 0.5, f"n={n}", ha="center", fontsize=9)

#Tasa por grupo edad (Panel derecho)
```

```
tasa_grupo_edad["tasa_%"].plot(kind="bar", ax=ax[1], color="#C44E52",
edgecolor="black")
ax[1].set_title("Tasa de suscripción por Grupo de Edad", fontsize=12)
ax[1].set_ylabel("Tasa de éxito (%)")
ax[1].tick_params(axis="x", rotation=45)
for i, (tasa, n) in enumerate(zip(tasa_grupo_edad["tasa_%"],
tasa_grupo_edad["n"])):
    ax[1].text(i, tasa + 0.5, f"n={n}", ha="center", fontsize=9)

plt.tight_layout()
plt.savefig("chart1.png", dpi=110)
plt.show()
```

Salida de gráfica



Paso 5 — Búsqueda de proxies

```
crosstab_job_marital = pd.crosstab(df_limpio["job"],
df_limpio["marital"], normalize="index") * 100

plt.figure(figsize=(8, 8))
sns.heatmap(crosstab_job_marital, annot=True, fmt=".1f",
cmap="coolwarm", cbar_kws={"label": "Porcentaje (%)"})
plt.title("Relación entre Ocupación y Estado Civil (Detección de
variable Proxy)", fontsize=13)
plt.xlabel("Estado civil")
plt.ylabel("Ocupación")
plt.tight_layout()
plt.savefig("chart2.png", dpi=110)
plt.show()
```

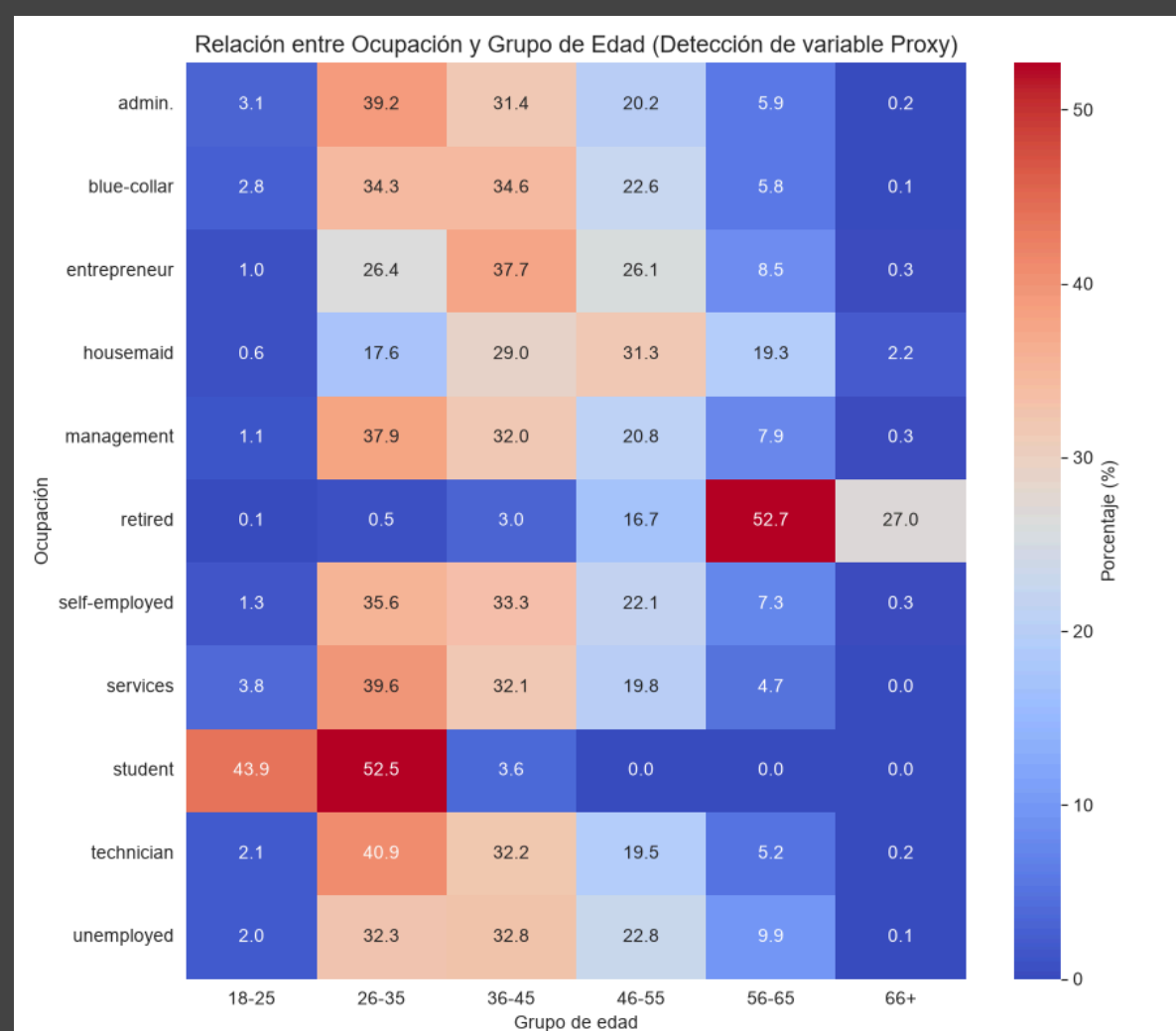
```

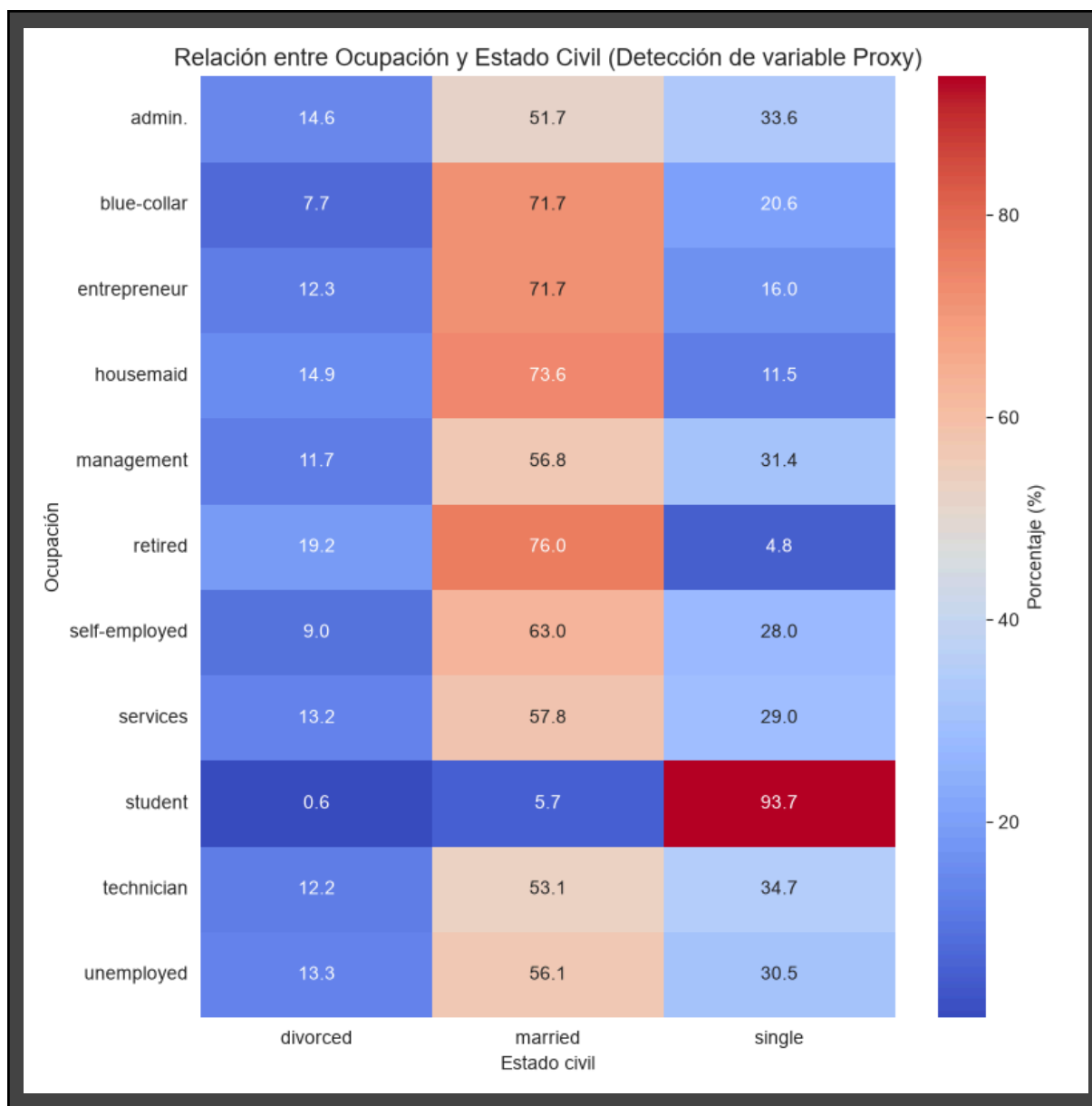
crosstab_job_edad = pd.crosstab(df_limpio["job"],
df_limpio["grupo_edad"], normalize="index") * 100

plt.figure(figsize=(9, 8))
sns.heatmap(crosstab_job_edad, annot=True, fmt=".1f",
cmap="coolwarm", cbar_kws={"label": "Porcentaje (%)"})
plt.title("Relación entre Ocupación y Grupo de Edad (Detección de
variable Proxy)", fontsize=13)
plt.xlabel("Grupo de edad")
plt.ylabel("Ocupación")
plt.tight_layout()
plt.savefig("chart3.png", dpi=110)
plt.show()

```

Salida de consola





4. ¿Hay ocupaciones dominadas casi por completo por un solo grupo (estado civil/edad)? Si esa variable job se usa en un modelo, ¿reintroduce el sesgo que marital/age ya representaban?

Sí, de forma muy marcada en dos categorías. student es 93.7% single y, cruzado con edad, 96.4% tiene entre 18 y 35 años casi un sinónimo categórico de "joven y soltero" en este dataset. retired es 76.0% married y 79.7% tiene 56 años o más casi un sinónimo de "mayor y casado". Esto confirma que job actúa como proxy fuerte tanto de marital como de grupo_edad: si se excluyeran marital y age de un modelo para "evitar sesgo" pero se conservara job, el modelo seguiría capturando indirectamente casi toda la señal de edad y estado civil que generó las disparidades del Paso 3, incluida la brecha de 33.83 puntos por edad. Es el mismo patrón de "fairness through unawareness" fallido que en el ejercicio de Adult Income, aquí con job como el proxy en vez de occupation.

Paso 6 — Reflexión escrita

5. ¿Qué variable(s) sensible(s) identificó y qué disparidad encontró (con el número exacto de la métrica calculada)?

Se identificaron dos variables sensibles: marital (estado civil) y age, agrupada en grupo_edad. La disparidad de paridad estadística fue de 4.86 puntos porcentuales (0.0486) para estado civil (single 14.89% vs. married 10.03%) y de 33.83 puntos porcentuales (0.3383) para grupo de edad (66+ con 43.13% vs. 36 - 45 con 9.30%). Solo la disparidad por edad supera el umbral de alerta de 0.10 - 0.20 discutido en la pregunta 3; la de estado civil queda por debajo.

6. ¿La limpieza aplicada (nulos, duplicados, outliers) alteró la proporción de algún grupo? ¿Cómo lo verificó?

Se verificó con la tabla comparación del paso 2: `value_counts(normalize=True)` de marital antes y después de convertir 'unknown' en job/education a NaN y aplicar `dropna(subset=cols_faltantes_genuinos)`.

El efecto fue mínimo (married -0.12 pts, divorced +0.12 pts, single sin cambio), a pesar de eliminar 2.018 filas (4.46% del total). No se verificó el impacto sobre grupo_edad de la misma manera (sería una mejora, como se señaló en la revisión de código), ni se comprobaron duplicados exactos con `df.duplicated().sum()` ambas son extensiones sencillas y recomendables antes de dar la limpieza por completa. Tampoco se trató contact ni poutcome pese a tener 28.80% y 81.75% de 'unknown' respectivamente, una decisión correcta y ya justificada (Paso 2) porque ese 'unknown' es estructural, no faltante al azar.

7. ¿Encontró alguna variable proxy? Argumente con evidencia del crosstab o correlación.

Sí, job es un proxy fuerte tanto de marital como de grupo_edad. La evidencia es directa en los dos crosstabs del Paso 5: student concentra 93.7% en single y 96.4% en el rango 18-35; retired concentra 76.0% en married y 79.7% en el rango 56+. Usando el mismo umbral de referencia del ejercicio anterior (una categoría de la variable no protegida por encima de 80 - 85% de una sola clase de la variable sensible), student y retired lo superan claramente en ambos cruces.

8. Proponga una mitigación concreta (no genérica) para este dataset específico.

Tratar age como variable continua con splines o age en vez de bins fijos de 10 años: la disparidad de 33.83 puntos está inflada por dos categorías pequeñas y muy heterogéneas (18 - 25 con n = 1231, 66+ con n = 670); un modelo continuo o con bootstrap de intervalos de confianza por grupo evita sacar conclusiones sobredimensionadas sobre esas colas.

Si job se usa como predictor, aplicar reweighing o auditoría estratificada tomando marital y grupo_edad como atributos sensibles conjuntos (no solo age), dado que job replica la señal de ambas variables a la vez en las categorías student y retired.

Para contact (28.80% unknown) y poutcome (81.75% unknown): en vez de imputar o eliminar, codificarlas explícitamente como "sin contacto previo" (ya es lo que representan) y evaluar si esa falta de contacto histórico correlaciona con age o marital podría ser un proxy adicional no explorado aún en este taller.

Antes de entrenar cualquier modelo, agregar df.duplicated().sum() y una comparación explícita de la distribución de age (no solo marital) antes - después de dropna(), para cerrar la verificación de impacto de limpieza que quedó parcial en el Paso 2.

Resumen de métricas calculadas

Métrica	Estado civil (marital)	Grupo de edad
Grupo con mayor tasa de suscripción	single — 14.89%	66+ — 43.13%
Grupo con menor tasa de suscripción	married — 10.03%	36-45 — 9.30%
Disparidad de paridad estadística	0.0486 (4.86 pts)	0.3383 (33.83 pts)
¿Supera el umbral de alerta (0.10–0.20)?	No	Sí, ampliamente
Filas eliminadas por limpieza (job/education)	2018 de 45211 (4.46%)	Igual (limpieza es global, no por edad)
Variable proxy detectada	job → student 93.7% single	job → retired 79.7% en 56+; student 96.4% en 18-35